



SISTEMAS OPERATIVOS - TEORÍA
13 de Junio de 2025

Nombre _____ DNI _____
Apellidos _____ Grupo _____

Cuestión 1 (2 puntos). Suponga un sistema de ficheros tipo UNIX simplificado con bloques de 4 KiB y punteros a bloque y a i-nodo de 32 bits. El i-nodo contiene, además de los metadatos básicos (tipo, contador de enlaces, permisos, fechas, etc.), 8 punteros directos a bloques de datos, 1 puntero indirecto simple, y 1 puntero indirecto doble. Cada entrada de directorio ocupa 64 bytes y consiste en un nombre de fichero (hasta 60 caracteres, incluyendo el terminador nulo) y el número del i-nodo al que referencia.

Se pide:

a) (0.5 puntos) ¿Cuál es el tamaño máximo de fichero que puede representarse con esta estructura de i-nodo? Si un directorio llamado /backup contiene 1300 entradas (incluyendo . y ..), ¿cuántos bloques de datos ocupa este directorio como mínimo? ¿Cuántos niveles de indirección del i-nodo del directorio /backup se necesitarían para direccionar dichos bloques de datos? Muestre sus cálculos y justifique sus respuestas.

b) (1 punto) Considere un fichero /data/report-0001.txt. Suponga que inicialmente existen otras 63 entradas (incluyendo . y ..) en este directorio. Se realizan las siguientes operaciones en secuencia:

1. Se crea un enlace duro: `ln /data/report-0001.txt /data/current_report.txt`
2. Se crea un enlace simbólico: `ln -s /data/report-0001.txt /home/user/current_report.txt`
3. Se borra el fichero original: `rm /data/report-0001.txt`

Para cada una de las tres operaciones, detalle **breve y esquemáticamente** qué estructuras del sistema de ficheros se ven afectadas: i-nodos creados/modificados, bloques de datos reservados/modificados/liberados, y cambios en el mapa de bits.

`ln /data/report-0001.txt /data/current_report.txt`
i-nodos creados:

i-nodos modificados:

bloques reservados/modificados/liberados:

Cambios en el mapa de bits:

```
ln -s /data/report-0001.txt /home/user/current_report.txt
```

i-nodos creados:

i-nodos modificados:

bloques reservados/modificados/liberados:

Cambios en el mapa de bits:

```
rm /data/report-0001.txt
```

i-nodos creados:

i-nodos modificados:

bloques reservados/modificados/liberados:

Cambios en el mapa de bits:

c) (0.5 puntos) Suponga que el fichero `/data/current_report.txt` del apartado b) tiene un tamaño de 4196 bytes. ¿Cuántos bytes se desperdician debido a la fragmentación interna? Explique **brevemente** el concepto de fragmentación interna en este contexto. ¿Se puede producir fragmentación externa? ¿Cómo?

Cuestión 2 (2 puntos). Un sistema utiliza un planificador con colas multinivel con realimentación con dos colas: una de alta prioridad gestionada por Round Robin con cuanto = 2, y otra de baja prioridad con política FCFS. Los procesos se mueven de la cola de alta prioridad a la de baja prioridad si agotan su cuanto. Siempre que terminan una operación de E/S los procesos se insertan en la cola de alta prioridad.

- a) **(1 punto)** Complete la traza de ejecución del planificador mostrada abajo para los procesos indicados en la tabla. Marque con X1 o X2 los ciclos en los que el proceso está ejecutando desde el nivel de prioridad 1 o 2 respectivamente. Marque con O los ciclos en los que cada proceso está esperando por la terminación de una operación de Entrada/Salida, y con -- cuando el proceso esté listo esperando a ejecutar (en alguna de las dos colas). En las filas de L1/L2 indique el estado de las colas de espera en cada uno de los ciclos, marcando el orden de los procesos que están esperando en dicha cola.

LEYENDA		PROCESO	INSTANTE DE LLEGADA	RÁFAGAS DE CPU Y E/S
SÍMBOLO	USO	P0	0	CPU(3), IO(2), CPU(2)
X1	CPU L1	P1	1	CPU(2), IO(3), CPU(3)
X2	CPU L2	P2	2	CPU(4), IO(2), CPU(1)
O	E/S	P3	3	CPU(3), IO(2), CPU(2)
--	LISTO			

Px/Tmpo.	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
P0																							
P1																							
P2																							
P3																							
COLA L1																							
COLA L2																							

- b) **(0.5 puntos)** Una vez completada la traza, calcule el tiempo de espera de cada proceso y el porcentaje total de uso de la CPU.
- Porcentaje de uso de CPU: _____ %
- Tiempo de espera de P0: _____ unidades
- Tiempo de espera de P1: _____ unidades
- Tiempo de espera de P2: _____ unidades
- Tiempo de espera de P3: _____ unidades
- c) **(0.5 puntos)** Con el planificador descrito anteriormente, ¿podría un proceso transitar directamente del estado **Bloqueado al estado de Ejecución**? Justifique su respuesta basándose en el modelo de estados de un proceso.

Cuestión 3 (2 puntos). En un sistema GNU/Linux, se ejecuta el siguiente programa que simula un antivirus. El programa genera varios procesos que escanean diferentes directorios del sistema. Cada proceso hijo lanza varios hilos, y cada hilo escanea un fichero individual. Los escaneos se registran en un fichero llamado log_output.txt. El código es el siguiente:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <fcntl.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>

#define NUM_DIRECTORIES 3
#define FILES_PER_DIR 4
int output_fd;

void* scan_file(void* arg) {
    int file_id = *(int*)arg;
    char log[64];
    sprintf(log, "[Thread] Scanning file ID %d\n", file_id);
    write(output_fd, log, strlen(log));
    sleep(1); // Simulate scanning time
    sprintf(log, "[Thread] File ID %d is clean\n", file_id);
    write(output_fd, log, strlen(log));
    return NULL;
}

int main() {
    pthread_t threads[FILES_PER_DIR];
    int file_ids[FILES_PER_DIR];
    char buffer[64];

    for (int i = 0; i < NUM_DIRECTORIES; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            output_fd = open("log_output.txt", O_WRONLY | O_CREAT | O_APPEND, 0644);
            sprintf(buffer, "[Process %d] Scanning directory %d...\n", getpid(), i);
            write(output_fd, buffer, strlen(buffer));

            for (int j = 0; j < FILES_PER_DIR; j++) {
                file_ids[j] = 1000 * i + j;
                pthread_create(&threads[j], NULL, scan_file, &file_ids[j]);
            }

            for (int j = 0; j < FILES_PER_DIR; j++) {
                pthread_join(threads[j], NULL);
            }

            sprintf(buffer, "[Process %d] Finished directory %d.\n", getpid(), i);
            write(output_fd, buffer, strlen(buffer));
            close(output_fd);
            exit(0);
        }
    }
    while (wait(NULL) > 0);
    return 0;
}
```

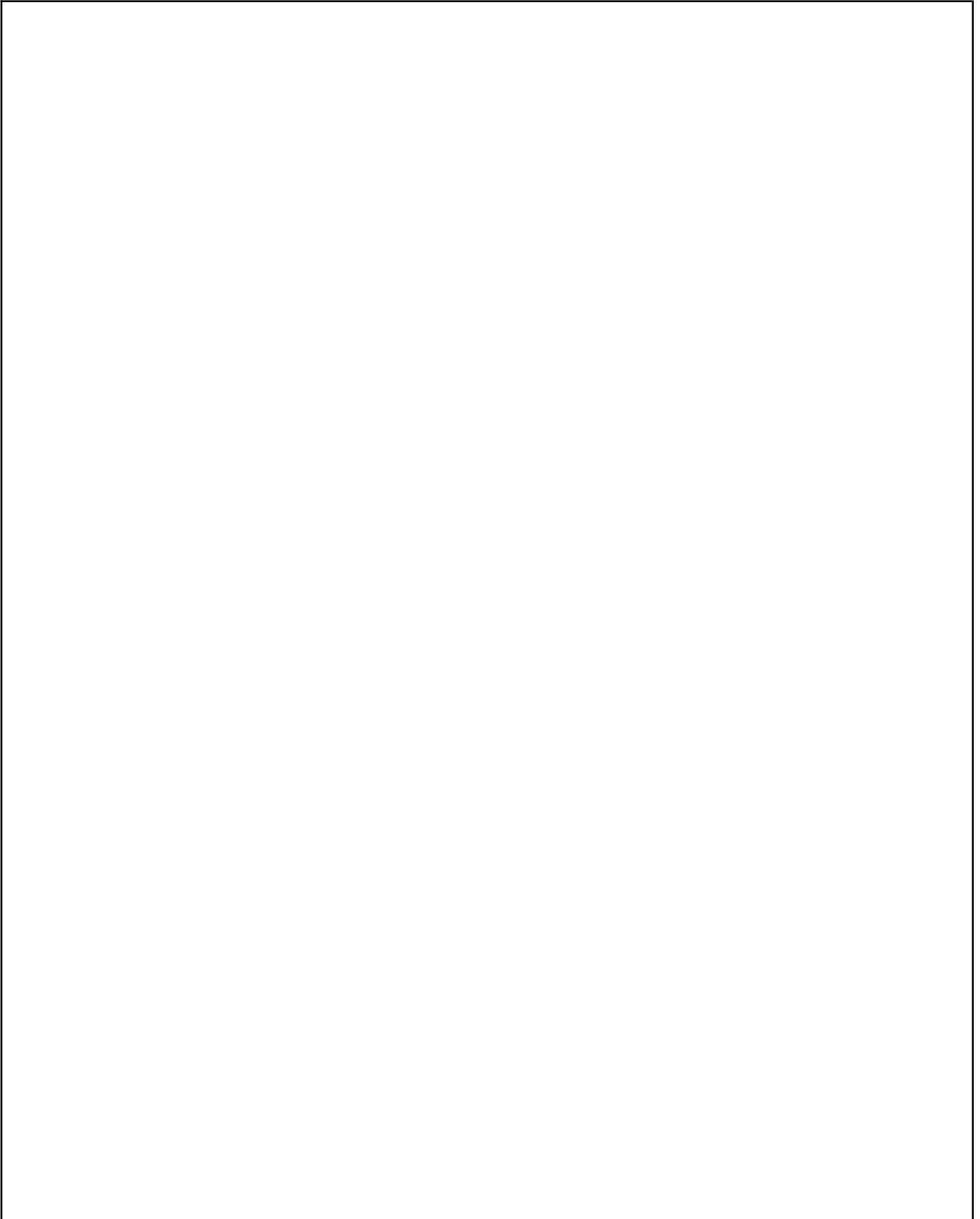
- a. ¿Cuántos procesos e hilos se crean durante la ejecución del programa? ¿Cuántos procesos/hilos se ejecutarán de forma concurrente como máximo?

- b. Dibuje el grafo de ejecución de los procesos e hilos. ¿Es posible conocer el orden exacto de ejecución de las operaciones? Justifique la respuesta.

- c. ¿Qué problemas podrían surgir al escribir de forma concurrente en el fichero log_output.txt? ¿Hay riesgo de pérdida de información?

- d. Indique las principales modificaciones que se deberían realizar en el código original para cumplir con la siguiente funcionalidad:

- a. Cada proceso hijo debe crear un fichero propio de registro log_output_X.txt, donde X es el índice del proceso (por ejemplo, log_output_0.txt para el proceso X=0).
- b. Cada hilo debe generar su salida de log en un buffer local. Una vez que todos los hilos hayan terminado, el proceso hijo debe escribir todo el contenido del buffer en el fichero, asegurando así que los logs de sus hilos queden ordenados y no se mezclen.
- c. El programa no debe utilizar mecanismos de sincronización entre procesos.



Cuestión 4 (2 puntos). En el sistema de gestión de la red eléctrica están planteando mejorar la aplicación de gestión de los generadores para evitar cortes de energía. El funcionamiento planteado es el siguiente:

- Cada generador tiene una potencia ***g_i***. Puede estar encendido (sumando a la generación total) o apagado.
- Tras cambios en la demanda (***d***), la generación total se tiene que adaptar para ser mayor o igual que ***d***. Sin embargo, este trabajo no está centralizado, sino que cada generador tiene que observar la red y:
 - Se enciende siempre y cuando observe que la demanda es mayor que la generación.
 - Se apaga cuando observe que su aportación es prescindible.

Se pide crear el código para los generadores (cada uno implementado mediante un hilo), para seguir este comportamiento, utilizando variables condicionales y mutex. Además, se debe crear la función que cambia la demanda. Ten especial cuidado con la sincronización, para evitar que se enciendan o apaguen más generadores de la cuenta.

```
typedef struct {  
    int id;  
    double potencia;  
    int encendido;  
} Generador;  
double demanda = 150.0;  
double generacion_total = 0.0;
```

```
/* Variables condicionales y mutex necesarios */
```

```
void* hilo_generador(void* arg) {  
    Generador* g = (Generador*) arg;  
    /* Lógica del generador */
```

```
}
```

```
void cambiar_demanda(double nueva_demanda) {  
    /* Actualización de demanda */
```

```
}
```

Cuestión 5 (2 puntos). Un sistema utiliza gestión de memoria virtual con paginación bajo demanda con páginas de 4KB, enteros y direcciones de memoria de 32 bits. Se ejecuta en dicho sistema el siguiente programa:

```
#include <stdio.h>
#include <stdlib.h>

#define VSIZE 4096

int A[VSIZE]={0};
int *B;

int main() {
    FILE *file = fopen("input.bin", "rb");
    int read_count, i;

    if (file == NULL) {
        perror("Error opening file");
        return 1;
    }
//A
    read_count = fread(A, sizeof(int), VSIZE, file);
    if (read_count != VSIZE) {
        fprintf(stderr, "Error: expected to read %d integers, got %zu\n", VSIZE, read_count);
        fclose(file);
        return 1;
    }

    fclose(file);

    B = (int *)malloc(VSIZE * sizeof(int));

    for (i = 0; i < VSIZE; i++)
        B[i] = 4*A[i];

    for (i = 0; i < VSIZE; i++)
        printf("%d\n", B[i]);

//B
    free(B);
    return 0;
}
```

Considerando que:

- Hay suficientes marcos de página libres en el sistema.
- El texto (código) del programa tras la compilación y enlazado estático ocupa 7,5KB.
- El contenido inicial de la pila al lanzar a ejecución el programa ocupa 32 bytes.

a. Indica cuántos fallos de página se producen entre los puntos marcados en el código como //A y //B, especificando claramente la sentencia que provoca cada uno, en qué tipo de región de memoria se produce el fallo, y cuántos fallos provoca cada una de esas sentencias.

b. Escribe un programa C que al ejecutarse en el sistema descrito genera una imagen de memoria con las regiones indicadas en la figura. Indica en qué punto de la ejecución de dicho programa la imagen de memoria del proceso correspondiente coincide con la de la figura, justificando brevemente por qué se crean las distintas regiones. Especifica también el tipo de enlazado del programa para conseguir dicha imagen de memoria, asumiendo que todas las funciones no definidas en el código se encuentran en una única biblioteca externa.

